Discrete Mathematics as a Precursor to Programming

Peter B. Henderson
Department of Computer Science
State University of New York at Stony Brook
Stony Brook, New York, 11794

1 Introduction

Computer programming courses are an option available to many high school and college students. These courses typically teach features of a specific programming language, and students develop programs to solve a variety of problems. Such courses build only slightly on prior mathematics experience, and do not necessarily reinforce mathematics skills. Discrete mathematics, however, provides an important link between mathematics, computing, and problem-solving. When and how should this link be made? What mathematics topics are most appropriate?

This paper describes preliminary experiences teaching an innovative discrete mathematics course for a wide range of student populations which include: high school students, college freshman planning to major in computer science, and mathematics education majors. The courses described are designed to address student weaknesses in discrete mathematics and problem-solving, and to illustrate the mathematical foundations of computing. They stress general problem-solving techniques and mathematical concepts that aid in the analysis and solution of computer based problems. The concentration is on general problem solving principles, patterns and symmetries, recursive and inductive thinking, algorithmic problem solving, and discrete mathematics concepts including: sets, propositional and predicate logic, proof techniques, relations, graphs and trees, functions, sequences, mathematical induction, language syntax, and algorithms. These concepts are motivated within the context of computer science, and its applications. For example, the principles of logic are used to understand logic programming, and functions are the underlying theory of applicative, or functional programming. To further enhance the learning experience, course material is reinforced through exploratory computer-based laboratory modules.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
© 1990 ACM 089791-346-9/90/0002/0017 $1.50

The courses described are based upon the *Foundations of Computer Science* course, which is the first course for computer science majors at The State University of New York at Stony Brook. *Foundations of Computer Science* has been taught for three years now. Preliminary material describing this course can be found in the literature [5,6]. The success of this foundations course at Stony Brook prompted us to try teaching similar courses at the high school level and for other groups of college students.

The prototype high school course was taught by Mr. John Thomas during the first half of the 1988-89 year to a select group of 18 students at West Islip High School on Long Island. Of these 18 students, 8 were sophomores, 6 were juniors, 4 were seniors; 14 were male, and 4 were female. The primary prerequisite was familiarity with a symbol based systems (e.g., algebra) and a willingness to learn. Accordingly, all students had completed one or two years of algebra. The course was intended to be taught prior to the traditional high school introductory computer science course.

The course *Mathematical Foundations of Computer Science* for mathematics education majors was taught by Sister Jane Fritz during the Fall 1989 semester at St. Joseph's College in Patchogue, N.Y. Most of the 20 students erolled in this course were juniors and seniors majoring in mathematics education. Some were planning to take an introductory programming course in the future. Again, the primary goal of this course was to convey basic mathematical concepts fundamental to computer science, to improve problem solving skills, and to develop mathematical maturity.

2 Overview of the courses.

One overly simplistic view of the computer science discipline portrayed in many computer courses is *Computer Science = Computer Programming.* The view established in these foundations courses is that problem solving and mathematics are fundamental to understanding computer science. An example may help to clarify this. There is an important relationship between the principle of mathematical induction and iterative (looping) struc-

tures in computer programs [9,10]. We know that students can learn to compose programs without first understanding mathematical induction. These students are being taught the **how to** rather than the **why** of computer science. They don't really understand the program development process, nor could they rigorously argue that the program they wrote correctly solves the specified problem.

Let's start by listing five primary objectives, and then describe ways in which these objectives can be attained in such foundations courses.

- Develop general problem solving skills, and provide a variety of representations for expressing problems, thinking about problems, and expressing solutions.
- Teach students fundamental mathematics concepts which can be used as tools for thinking and problem solving.
- Use these mathematical foundations to explain the underlying principles of new approaches for computer interaction. For example, logic programming (Prolog, Trilogy) and functional programming (Standard ML, Miranda).
- Have students gain an appreciation for mathematics and its relationship to computer science.
- Learn to use basic mathematical principles to understand algorithms and algorithmic problem solving.

2.1 General problem solving skills.

The first component of these courses focuses on problem solving. Most students have been exposed to problem solving in other courses, but few have concentrated on it. With the proper instruction, students can become better problem solvers. This is achieved by challenging students to get them actively involved in the problem solving process, emphasizing problem solving approaches rather than just getting the answer, and asking them to think about how they solve problems. There are numerous approaches to problem solving detailed in text books and monographs [18,15,2,8,11].

Some of the topics touched upon are: problem representation, inference, deduction, inductive reasoning, divide-and-conquer, contradiction, incubation, working backwards, and relationships between problems. Many of these topics are introduced early and then reinforced during the course. We found that having students work together in groups of two to four improved student participation and motivated a better appreciation for general problem solving skills. Groups emphasize active engagement, communication, and encourage trial and error.

A representative problem follows.

> A social psychologist was interested in the custom of handshaking. He noticed that some people are more inclined than others to shake hands when they are introduced. One evening when he and his wife had joined four other married couples at a party, he took advantage of the occasion to collect data. He asked each of the other nine people at the party how many people they had shaken hands with during the introductions. He received a different answer, zero through eight, from each of the nine people. You can assume that husbands and wives didn't shake hands with each other during introductions, and of course, people didn't shake hands with themselves. Given this information, find out how often the psychologist's wife shook hands.

This problem, which at first glance appears to be unsolvable, requires students to find an appropriate representation for the information presented[8].

2.2 Discrete mathematics concepts.

Discrete mathematics is the foundation for much of computer science, and provides invaluable tools for thinking and problem solving. For example, logic permits students to mathematically formulate and solve a wide class of problems, is fundamental to understanding the meaning of algorithms, and represents the foundations of logic programming, artificial intelligence and expert systems.

The text book *Discrete Structures: An Introduction to Mathematics for Computer Science* by Fletcher Norris is the primary source of instruction for basic discrete mathematics concepts for all these courses[13]. These concepts include propositional logic, sets, binary relations, functions, recursion, mathematical induction, and graphs and trees. Many of these concepts were further reinforced through supplemental material, exercises, and laboratory assignments. For instance, students symbolically encoded several logic problems to be used as input to an interactive theorem prover. Students learned the different strategies used by the prover to deduce the desired result.

2.3 Mathematics as a tool.

In many discrete mathematics courses, the focus is on understanding the concepts. We believe it is important to motivate these concepts in the context of representative problems which can be solved by properly applying one or more of the concepts. For example, simply teaching students how to prove various relationships using mathematical induction does not necessarily improve their inductive reasoning skills, or necessarily link inductive reasoning and mathematical induction in their minds. The following problem illustrates inductive reasoning by identifying and applying patterns.

> You know that a truth table with 2 logical variables has 4 rows, one with 3 variables has 8 rows,

etc.

i) If you knew that a truth table with $n-1$ logical variables has K rows, then in terms of K how many rows would a truth table with n logical variables have? Expressed in terms of n, how many rows would a truth table with n logical variables have?

ii) Assume you have constructed a truth table for $n-1$ logical variables, show how this table can be used to construct a truth table for n logical variables.

Other problems illustrate other important concepts. For example, the problem below demonstrates looking for, and formally specifying, invarient properties in problems. Here the invarient property is relatively simple – if the number of tails in the starting configuration is odd(even) then it will always be odd(even).

N coins are placed on a table, some with heads facing up, the others with tails facing up. You may flip any two coins at a time. Using a sequence of such flips, is it always possible to get all tails facing up? If so, prove it. If not, give the conditions under which it is unachievable.

2.4 New paradigms for computer interaction.

The traditional algorithmic based paradigms for computer programming are not easily amenable to mathematical analysis or proof. Accordingly, computer scientists are seeking new techniques and languages for interacting with computers that are based upon sound mathematical theories. An example is logic programming, an evolving paradigm based upon mathematical logic.

It is important for students to be exposed to the mathematics which forms the foundations of new programming paradigms for several reasons. They begin to see the important connection between mathematics and computer science. An understanding of and appreciation for different approaches to computer problem solving is acquired. By understanding the mathematics first, students find it easier to learn a new language. The mathematics will give students a glimpse into the future and make it easier for them to adapt to new techniques and languages.

Some representative examples are in order. Prolog, Trilogy[17], and many theorem proving techniques used in artificial intelligence are based on mathematical logic. General problem solving techniques (state action trees, subgoals, contradiction, and working backwards) which are used to search for solutions to problems are central to both logic programming and artificial intelligence. All relational database systems are dependent upon the mathematical theory of relations. Sets are the underlying vehicle for the very high level programming language SETL[7]. Functions and their underlying mathematical theory are crucial to a variety of important new functional programming languages including ML[1] and Miranda[16].

2.5 Appreciation for mathematics.

As noted in the introductory section, programming courses rarely build on prior mathematics knowledge, and mathematics is usually not stressed in such courses. Gaining an appreciation for the important relationship between mathematics and computer science is best achieved by exposing students to the mathematical concepts and showing them how those concepts are used in the framework of computer science. This is one objective of these courses. Specific points have been made earlier and will not be reiterated here.

2.6 Algorithmic problem solving

Algorithms are usually generalizations of specific problems to be solved. For example, finding the sum of 5, 2, 43, 7 and 12 generalizes to finding the sum of an arbitrary set of numbers. Algorithmic problem solving is learning to generalize patterns from specific problem instances and being able to precisely express these patterns. We have found that teaching students to think mathematically helps them to learn how to generalize and develops basic notions of abstraction necessary to express algorithmic solutions.

To understand and develop algorithms, students must feel comfortable with symbols and their manipulation. Symbols and symbol manipulation are integral components of mathematics; hence, through mathematics students can become more effective algorithmic problem solvers because they better understand the role of symbols in the development of an algorithm.

There are several mathematical concepts which are important for understanding algorithms and their development. At the beginning of this section, the relationship between induction and the iterative control structures of algorithms was noted[9,10]. Iterative control is one of the most complex aspects of algorithms to model and understand. Induction provides students with a tool for developing correct algorithms. It helps students understand, analyze, prove correct, and document algorithms. To illustrate, consider the algorithm fragment shown in Figure 1. The inductive hypothesis for the nth iteration of the **repeat** loop is: Assume we have computed the value of the polynomial $P_{n-1}(x)$, where

$$P_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \cdots + a_1$$

Mathematical logic provides students with a tool for precisely describing information about the state of an algorithm at each step. Logical assertions and invariant

$P := a_n$;
repeat n **times**
  $P := x \times P + a_{n-1}$;

Figure 1: Algorithm for Evaluating a Polynomial

assertions[3] permit students to formally express what an algorithm is doing. These are some of the notions required to use mathematics to accurately model the meaning of algorithms. Without them, the best we can hope for is an intuitive understanding of algorithms.

3 Pedagogy

Since the goals of these courses were different from those of the traditional mathematics or computer science course, teaching them required some unique approaches to instruction and a willingness to test new ideas. Fundamental issues included: motivating learning, actively involving students in the learning process, group dynamics, challenging students without demoralizing them, interweaving problem solving and other important concepts throughout the courses, dealing with notation and terminology, selection of appropriate material, and organization and scheduling of laboratories.

Several approaches were used to motivate learning. Class discussions encouraged students to get involved in problem solving activities. Students who were reluctant to participate in such discussions would frequently open up in small group sessions where the perceived peer pressure was diminished. These study groups got students communicating with each other, thus enhancing the learning experience for many of them. One of the keys to getting students working together is interesting, challenging problems. Such problems encourage interaction since they are fun, and typically difficult for individual students to solve on their own. In such an environment, students benefit from study groups, which may not always be the case in other courses.

Students were given a longer time (one or two weeks) to work on more challenging problems. They were expected to start on problems when first assigned since such problems usually couldn't be solved in one sitting. Accordingly, when they didn't understand a problem they were encouraged to ask questions, and they learned the value of putting a problem aside when stymied. Such a problem solving environment is more representative of the real world.

In most courses there are concepts which are traditionally difficult for students to grasp. In mathematics and computer science, recursion and induction are two such concepts. Conveying difficult concepts is best achieved slowly and patiently by interweaving them throughout the course. Start by motivating them intuitively with simple examples and problems. These simple mental models are continually reinforced through examples and problems related to the current topic of discussion. We have found this approach to be very effective.

Terminology and notation are often more frightening than the concept itself. Whenever possible strive to have students feel comfortable with a concept before using standard terminology. This can be achieved by introducing simple examples and problems from which students can generalize. This is especially important for the more difficult concepts. Most text books motivate an understanding of terminology and notation with examples. We feel this is backwards, and therefore counterproductive. The text used for this course attempts to administer a minimal dosage of terminology and notation[13].

4 Laboratories

Laboratory assignments are designed to reinforce basic concepts, and to provide hands on experience and motivation. They were developed at SUNY at Stony Book for the *Foundations of Computer Science* course. Other computer based approaches to teaching discrete mathematics require students to compose short programs in BASIC, Pascal or Interactive SETL[12]. These laboratory assignments are not computer programming assignments in this traditional sense, but rather computer-based instructional modules in which students explore ideas and concepts, and learn to use abstraction as an effective problem solving tool. We have devoted extra effort to trying to match the concept with the proper learning tool.

Current laboratory courseware includes: a truth table generator, discovering the difference between declarative and imperative styles of programming, identifying patterns in sequences, demonstrating logical proof techniques using an interactive theorem prover, logic programming, discovering fundamental information structures, problem solving with functions, algorithmic problem solving using Karel the Robot[14] and algorithm discovery courseware[4]. These laboratories run under Berkeley UNIX and most are available on the Apple Macintosh.

Unfortunately, due to different equipment (West Islip and St. Joseph's have IBM PCs) students at those locations could not run some of the labs. Some students were able to run labs, in a limited capacity, via a modem connection to Stony Brook. At West Islip students worked with the logic programming language Trilogy on IBM PCs, and devoted several weeks to problem solving with functions using the functional programming language Standard ML on borrowed Macintoshs. At St. Joseph's students used Interactive SETL[12] as the primary vehicle for understanding fundamental concepts. More details regarding these labortory assignments are available from the author on request. Also see the paper *Teaching Recursion as a Problem-Solving Tool Using*

Standard ML[6].

5 Evaluation and Assessment

Based on our impressions and initial student reaction these courses were very successful. They were non-traditional mathematics courses with an emphasis on student interaction and applications of mathematics to problem solving and computer science. Most students found this particularly appealing.

Almost universally the students enjoyed problem solving which was emphasized early in the courses and continued throughout. Many of the students indicated that they were not accustomed to participating openly in classroom discussions, however they did feel more comfortable working in smaller groups. Students always like working with computers. They commented that the laboratories helped them to better understand the course material, and complained there were not enough computer based assignments.

Not all students found the courses to their liking. The most common complaints included the difficulty of the course material. Some students obviously preferred a rote learning environment to a more challenging course and discussion oriented classroom .

We found that factors, such as prior computer experience, and differing levels of mathematics backgrounds had some, but not a major impact, on the performance of the students. The most pronounced difference was with the mathematics education majors. Students with a strong mathematics background experienced fewer difficulties than those with less formal training.

Teaching this course for the first time was difficult for both Mr. Thomas (West Islip High School) and Sister Jane (St. Joseph's College). Although familiar with the material, Mr. Thomas had not taught a discrete mathematics course before. Both found the fundamental pedagogical issues presented in Section 3 to be sound in theory, but required innovative approaches in practice. For instance, motivating concepts with examples is highly dependent upon the choice of examples. Simply scaling down the examples and problems from the rigorous Stony Brook course was not always possible. This frequently required revising existing problems or devising new problems. Maintaining active classroom discussion was a challenge at the high school level. The laboratories posed a unique problem due to hardware and software incompatibilities (Stony Brook uses UNIX on VAX and Macintoshes, West and St. Joseph's have IBM PCs). These issues very likely lead to the perceived lack of cohesiveness by several students.

References

[1] Åke Wikström. *Functional Programming using ML.* Prentice-Hall International, January 1987.

[2] A. Averbach and O. Chein. *Mathematics: Problem Solving Through Recreational Mathematics.* W.H. Freeman, 1980.

[3] David Gries. *The Science of Programming.* Springer-Verlag, 1981.

[4] Peter B. Henderson. A computer-aided instructional environment. *The Journal of Educational Technology Systems*, 15(3):281–302, January 1987.

[5] Peter B. Henderson. Modern introductory computer science. *ACM SIGCSE Bulletin*, 18(1):183–189, February 1987.

[6] Peter B. Henderson and Francisco Romero. Teaching recursion as a problem-solving tool using standard ml. *ACM SIGCSE Bulletin*, 20(1):27–30, February 1989.

[7] E. Dubinsky J.T. Schwartz, R.B.K. Dewar and E. Schonberg. *Programming with Sets: An Introduction to SETL.* Springer-Verlag, 1986.

[8] Marvin Levine. *Principles of Effective Problem Solving.* Prentice-Hall, 1988.

[9] Udi Manber. *Introduction to Algorithms - A Creative Approach.* Addison-Welsley, 1989.

[10] Udi Manber. Using induction to design algorithms. *CACM*, 31(11):1300–1313, November 1988.

[11] J. Mason and L. Burton. *Thinking Mathematically.* Addison-Wesley, 1982.

[12] E. Dubinsky N. Baxter and G. Levine. *Learning Discrete Mathematics with ISETL.* Springer-Verlag, 1988.

[13] Fletcher R. Norris. *Discrete Structures: An Introduction to Mathematics for Computer Science.* Prentice-Hall, 1985.

[14] Richard E. Pattis. *Karel the Robot.* John Wiley and Sons, 1981.

[15] Moshe F. Rubinstein. *Tools for Thinking and Problem Solving.* Prentice-Hall, 1986.

[16] D.A. Turner. *Miranda: A Non-strict Functional Language with Polymorphic Types.* Springer Lecture Notes in Computer Science, 1985.

[17] Paul Voda and Jamie Andrews. *TRILOGY Users Manual.* Complete Logic Systems, Inc., 741 Blueridge Ave., North Vancover, B.C. V7R2J5, Canada, 1987.

[18] Wayne Wickelgren. *How to Solve Problems.* W.H. Freeman and Co., 1974.